# Game Design Document - Lighthouse of Alexandria

## 1. Project Identity

- Title: Lighthouse of Alexandria

- Genre: top-down 2D adventure with educational circuit-analysis puzzles

- Current platform: Windows desktop

- Engine and language: Python with ‘pygame-ce‘

- Project type: narrative educational game with a complete single-player campaign

## 2. High Concept

Lighthouse of Alexandria is a story-driven 2D game where the player explores hostile maps, talks to characters, gathers electrical components, and solves circuit panels that gate progression. The same systems that teach electricity also drive tension, combat pressure, and narrative stakes.

The player controls Kevin, who becomes involved in a conflict around the Lighthouse of Alexandria, time-line manipulation, and a painful family decision. The game uses electrical reasoning not as a detached quiz, but as the main way the player advances through the world.

## 3. Vision Statement

The project aims to merge three forms of engagement into one coherent experience:

- spatial exploration in top-down levels;

- dramatic motivation through narrative scenes and dialogue;

- structured learning through circuit construction and validation.

The core design promise is simple: learning the system should help the player survive, progress, and understand the story.

## 4. Design Goals

- Make circuit reasoning feel like a meaningful gameplay verb, not a separate worksheet.

- Deliver a full campaign with rising narrative and mechanical stakes.

- Introduce electrical concepts in a staged learning curve.

- Balance educational challenge with map pressure, enemies, and hazards.

- Support both Portuguese and English throughout gameplay-facing content.

## 5. Player Fantasy

The player fantasy is to feel like an explorer-engineer solving real problems under pressure. Kevin is not just traversing rooms and defeating enemies; he is restoring systems, interpreting technical situations, and preventing a historical catastrophe.

## 6. Audience

Primary audience:

- students learning introductory electricity and circuit analysis;
- teachers and researchers interested in educational games;
- players who enjoy puzzle-heavy adventure games with narrative context.

Secondary audience:

- academic showcases and capstone/demo environments;
- players interested in lightweight action mixed with technical problem solving.

## 7. Core Pillars

### 7.1 Circuits Are The Main Progression System

Panels, locks, and key interactions depend on circuit assembly and validation. Circuits are not optional side content.

### 7.2 Learning Through Escalation

Each major learning topic is introduced in an explanation scene and then applied in a dangerous playable level.

### 7.3 Narrative And Mechanics Reinforce Each Other

The story creates a reason to solve each problem, while the puzzle systems create the means to move the story forward.

### 7.4 Pressure Makes Knowledge Matter

The game asks the player to think while navigating enemies, hazards, limited safety windows, and spatial routing.

## 8. Game Structure

The current campaign flow is:

1. Main menu
2. Intro home scene
3. Level 1
4. Level 2
5. Explanation stage 3
6. Generic level 3
7. Explanation stage 4
8. Generic level 4
9. Explanation stage 5

10. Generic level 5

11. Explanation stage 6

12. Generic level 6

13. Explanation stage 7

14. Generic level 7

15. Final level

16. Lighthouse rekindle ending scene

17. Home-after ending scene

18. Ending thanks and credits

This makes the project a complete campaign rather than a narrow prototype.

## 9. Core Gameplay Loop

The dominant loop inside puzzle-combat levels is:

1. Explore the map

2. Read the situation through dialogue, layout, and interactables

3. Gather required components

4. Reach a panel or objective

5. Open the circuit editor

6. Assemble and validate a circuit

7. Trigger an environmental or progression change

8. Survive enemies and hazards

9. Reach the next objective or exit

This loop combines observation, routing, technical reasoning, and execution.

# 10. Controls

## 10.1 Exploration

- ‘W A S D‘: movement

- ‘E‘: interact

- ‘B‘: place bomb in stages that use the bomb/core system

- ‘L‘: toggle light in stages that support it

- ‘Esc‘: menu

- ‘F1‘: help

## 10.2 Circuit Editor

- ‘N‘: node

- ‘W‘: wire

- ‘G‘: GND

- ‘R‘: rotate

- ‘S‘: select

- ‘Delete‘: delete tool

- ‘Esc‘: cancel current tool

The game also includes controller-oriented support described in the help scene.

# 11. Main Systems

## 11.1 Exploration, Collision, And Scene Progression

The player moves through top-down maps with collision, contextual prompts, doors, and scene transitions. Camera and scene transitions support the pacing between puzzle and narrative beats.

## 11.2 Dialogue And Story Delivery

NPCs, old papers, scripted areas, and ending scenes communicate story context. Dialogue can unlock progression, explain stakes, and orient the player toward puzzle goals.

## 11.3 Circuit Editor

The circuit editor is the central gameplay interface. It supports:

- resistors;

- voltage sources;

- current sources;

- nodes;

- wires;

- GND.

Players can place, rotate, connect, save, clear, load, and solve configurations tied to specific panel files.

### 11.4 Circuit Validation

The game validates player-built circuits against expected challenge conditions. Current validator coverage includes:

- resistor electrical quantities;

- resistor association and reduction logic;

- pair and relation-based resistor checks;

- Thevenin and Norton equivalent validation;

- maximum power transfer validation.

### 11.5 Component Collection And Storage

Map exploration feeds the editor. Components picked up in the world are stored and later used in puzzles, linking spatial progression and technical problem solving.

### 11.6 Bomb/Core System

Some stages include a bomb-core subsystem with its own editor and calibration flow. This creates an alternate puzzle pressure model where electrical setup changes combat or stage interaction outcomes.

### 11.7 Enemies And Threat Systems

The campaign includes multiple pressure systems:

- phantoms with radial proximity-based lethality;
- spiders and web ambush/slow mechanics;
- map hazards such as fall-ground traps;
- death and respawn flows tied to stage logic.

Recent fairness work now makes phantoms respawn at their original spawns, uses a dedicated chase-start sound, and keeps fall-ground traps readable through warning audio and a short arm delay.

### 11.8 Light And Visibility

Some stages use permanent or temporary light as both atmosphere and gameplay state. Light can affect readability, mood, and player planning.

### 11.9 Life, Death, And Retry Flow

The game uses a life manager, death transition handling, and scene-specific re-entry rules. Failure can restart sections while preserving the broader campaign state flow.

### 11.10 Save And Preferences

The save system tracks:

- current scene;

- current lives;

- maximum lives.

The game also stores preferences such as language selection. Runtime data for packaged builds is copied to the user profile under ‘%LOCALAPPDATA%‘.

## 12. Educational Progression

### 12.1 Level 3 Topic

- Ohm’s law

- electric power

- ground reference

- reading voltage, current, and power relationships

### 12.2 Level 4 Topic

- series association

- parallel association

- mixed reduction

- voltage division

### 12.3 Level 5 Topic

- Kirchhoff’s laws

- nodal reasoning

- circuit equation setup

### 12.4 Level 6 Topic

- Thevenin equivalent

- Norton equivalent

- deriving ‘Vth‘, ‘Rth‘, ‘In‘, and related values

### 12.5 Level 7 Topic

- maximum power transfer

### 12.6 Final Level Topic

- applying learned concepts under pressure, with timing and route efficiency

# 13. Level Roles

## 13.1 Home Scene

Narrative opening and emotional grounding.

## 13.2 Levels 1 And 2

Onboarding for movement, interaction, doors, key objects, and world logic.

## 13.3 Explanation Scenes

The explanation scenes present theory, visual guidance, and concept framing before each applied challenge.

## 13.4 Generic Levels 3 To 7

These are the main educational-action stages, combining:

- component search;

- panel solving;

- enemy avoidance;

- stage-specific environmental pressure.

### 13.5 Final Level

The final level acts as the campaign exam, requiring sustained execution and repeated panel handling with high pressure.

### 13.6 Ending Scenes

The final scenes resolve the symbolic and emotional outcome of the campaign and close the player journey.

## 14. Narrative Summary

Kevin learns that the Lighthouse of Alexandria sits at the center of a disastrous timeline decision. His father intends to alter a key historical outcome in a way that may save Kevin’s grandmother but destroy Alexandria. The conflict places personal loss, moral responsibility, and technical action in direct collision.

The story tone combines:

- historical fantasy;

- family drama;

- technical problem solving;

- urgency around irreversible consequences.

# 15. Main Characters

### 15.1 Kevin

Playable protagonist. He represents the bridge between technical reasoning and personal stakes.

### 15.2 Kevin’s Father

Tragic antagonist. His goal is emotionally understandable even when the consequences are catastrophic.

### 15.3 Archimedes

Guide and support figure. He helps anchor both the intellectual and narrative dimensions of the campaign.

# 16. Localization And Accessibility Support

The current game supports Portuguese and English across:

- main menu labels;

- help scene content;

- UI prompts;

- dialogue and story text;

- letters;

- explanation-stage media and panel-support assets.

Language can be toggled from the main menu, and the chosen preference is persisted.

## 17. UX And Feedback

Current player-support layers include:

- context prompts for interaction;

- help scene with controls and editor guidance;

- life display;

- bomb status;

- stealth timer and overload indicators where relevant;

- dedicated audio feedback for phantom chase starts;

- dedicated warning audio for fall-ground activation.

These systems help the player understand danger, state changes, and available interactions.

## 18. Technical Architecture

The project uses:

- Python and ‘pygame-ce‘;

- a custom ECS-style architecture;

- map-driven scene construction via ‘PyTMX‘;

- circuit solving support built around ‘sympy‘, ‘numpy‘, and ‘pandas‘;

- scene registration and transition management through central managers.

Automated tests cover localization flows, fall-ground fairness behavior, phantom behavior, and related gameplay regressions.

## 19. Current Production Status

The game is currently a playable end-to-end campaign with:

- a complete scene flow from intro to ending;

- multiple explanation and applied puzzle stages;

- an integrated circuit editor;

- specialized validation logic for several electricity topics;

- two-language support;

- Windows build and itch.io packaging workflows;

- automated test coverage for critical systems.

## 20. Known Design Challenges

### 20.1 Cognitive Load

The game asks the player to read, move, evade danger, and reason about circuits, sometimes in the same stage. This is a strength, but it also requires careful pacing.

### 20.2 Balancing Puzzle Difficulty And Spatial Pressure

Enemy aggression, map layout, retries, and technical difficulty must stay aligned so the game feels demanding without becoming unreadable.

### 20.3 Teaching Clarity

The player must be able to tell whether failure came from:

- a wrong concept;
- a wrong circuit layout;
- a wrong component value;
- poor route execution under pressure.

## 21. Opportunities For Future Work

- continue balancing bomb-heavy and enemy-heavy stages;
- expand explanation media and teaching support;
- strengthen release automation for executable and zip generation;

- add more user-test findings to guide pacing and fairness improvements;

- append per-level validator notes and puzzle acceptance criteria.

## 22. Conclusion

Lighthouse of Alexandria is a full educational adventure game that turns electrical reasoning into a real dramatic and mechanical progression system. Its strongest differentiator is the way circuit analysis, environmental pressure, and narrative motivation reinforce each other across a complete campaign.

As it stands, the project is already suitable both as a playable game and as an academic artifact for discussion around educational game design, technical content integration, and system-driven narrative progression.